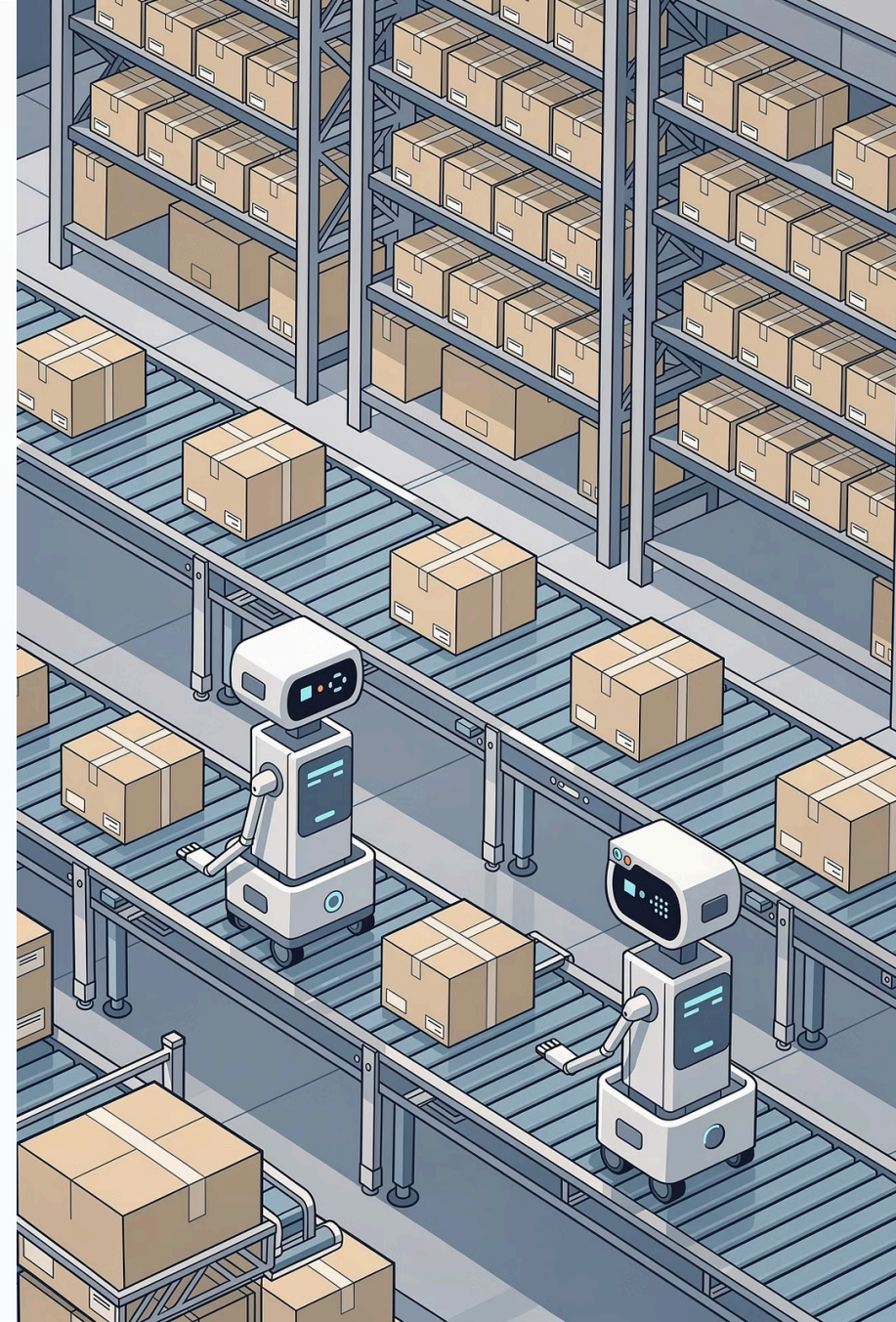


PrimeFlow Fulfillment Hackathon

STACKS & QUEUES · PROBLEM STATEMENTS

Developed by **TalenciaGlobal**



About This Hackathon

PrimeFlow is the dispatch and logistics core of an Amazon-style fulfillment network. Across eight levels you will build it up: a queue from two stacks, a min-stack price tracker, a sliding-window demand monitor, a stock-span demand forecaster, a circular event buffer, a round-robin scheduler, a warehouse-robot pathfinder, and a delivery-reach analyzer.

- Each level states the inputs, the required outputs and the rules you may rely on. The levels run from intermediate to expert and apply stacks and queues to real fulfillment problems. No starter code is provided — design and write each solution yourself.



Levels

1	Level 1 Queue From Two Stacks ★★★ INTERMEDIATE
2	Level 2 Min Stack Price Tracker ★★★ INTERMEDIATE
3	Level 3 Demand Spike Monitor ★★★★ ADVANCED
4	Level 4 Demand Forecaster ★★★★ ADVANCED
5	Level 5 Circular Event Buffer ★★★★ ADVANCED
6	Level 6 Robot Scheduler ★★★★ ADVANCED
7	Level 7 Warehouse Pathfinder ★★★★★ ADVANCED-EXPERT
8	Level 8 Delivery Reach ★★★★★ EXPERT

Level 1 — Queue From Two Stacks

PrimeFlow's intake must behave as a FIFO queue, but the only primitive available is a LIFO stack. Build a working queue from two stacks: an **inbox** you push onto, and an **outbox** that reverses the order on demand so the oldest item comes out first.

Concepts Covered

- Implementing a queue with two stacks
- Lazy transfer between stacks
- Amortized $O(1)$ dequeue
- Preserving FIFO order from LIFO parts

Problem Statement

Given: A count n , then n operations: ENQUEUE with an id, or DEQUEUE.

Required output: The ids served, in FIFO order, and the number of items still in the queue.

Constraints: ENQUEUE pushes onto the inbox stack. DEQUEUE pops from the outbox; if the outbox is empty, first move every inbox item across (reversing it). A DEQUEUE on an empty queue serves nothing.

Formula Guidance

```
enqueue -> inbox.push(id)
dequeue -> if outbox empty: move all inbox into outbox; then outbox.pop()
```

Sample Input

```
5
ENQUEUE 1
ENQUEUE 2
DEQUEUE
ENQUEUE 3
DEQUEUE
```

Sample Output

```
Served: 1 2
Pending: 1
```

⚠ **Common Pitfall:** Refill the outbox only when it is empty. Transferring while the outbox still holds items would scramble the order.

💡 **Thinking Skill:** Reversing one stack into a second stack flips LIFO into FIFO; doing it lazily — only when the outbox runs dry — keeps the amortized cost constant.

Level 2 — Min Stack Price Tracker

PrimeFlow tracks the lowest price in a stack of supplier quotes as quotes are pushed and popped. A fresh scan for the minimum is $O(n)$ per query; instead keep a parallel stack so the current minimum is always available in **$O(1)$** .

Concepts Covered

- Augmenting a stack with extra state
- A parallel stack of running minima
- $O(1)$ minimum queries
- Keeping two stacks in lock-step

Problem Statement

Given: A count n , then n operations: PUSH with a price, POP, or MIN.

Required output: For each MIN, the current minimum price (or EMPTY). Finally, the number of prices left on the stack.

Constraints: PUSH adds a price; POP removes the top (and does nothing if empty). MIN reports the smallest price currently on the stack, or EMPTY. Prices are integers and may repeat.

Formula Guidance

on PUSH x : `mins.push(min(x, current minimum))`
on POP: pop both stacks
MIN = top of the mins stack

Sample Input

8
PUSH 5
PUSH 3
MIN
PUSH 7
MIN
POP
POP
MIN

Sample Output

Min: 3
Min: 3
Min: 5
Size: 1

 **Common Pitfall:** Push and pop the mins stack in lock-step with the data stack; if their heights ever diverge, the reported minimum goes wrong.

 **Thinking Skill:** Carrying a running minimum alongside each element turns an $O(n)$ query into an $O(1)$ read — a classic trade of a little space for a lot of time.

Level 3 — Demand Spike Monitor

PrimeFlow watches page-view spikes over a moving window of days. For every window of size k , report the peak. A **monotonic deque** of indices yields each window maximum in amortized $O(1)$.

Concepts Covered

- A double-ended queue of indices
- Maintaining a decreasing deque
- Evicting indices that fall out of the window
- $O(n)$ sliding-window maxima

Problem Statement

Given: A length n and a window size k , then n integers.

Required output: The maximum of each window of k consecutive values, left to right.

Constraints: $1 \leq k \leq n$. The deque holds indices whose values are useful (decreasing front to back). Each index enters and leaves the deque at most once.

Formula Guidance

front of deque = index of the current window's maximum
evict the front when its index $\leq i - k$
evict the back while $a[\text{back}] \leq a[i]$

Sample Input

```
8 3
1 3 -1 -3 5 3 6 7
```

Sample Output

Window Maxima: 3 3 5 5 6 7

⚠ **Common Pitfall:** Store indices, not values, so you can tell when the maximum has slid out of the window — and evict from the front before reading it.

💡 **Thinking Skill:** Keeping the deque monotonic means the answer is always at the front; because each index is pushed and popped once, the whole sweep is linear.

Level 4 — Demand Forecaster

For each day's demand, PrimeFlow wants to know how many days pass until demand is strictly higher — a stock-span-style forecast. A **monotonic stack** of waiting days answers all of them in a single pass.

Concepts Covered

- A stack of indices awaiting resolution
- A decreasing monotonic stack
- Resolving many days from one higher day
- $O(n)$ next-greater-element

Problem Statement

Given: A length n , then n demand values.

Required output: For each day, the number of days until a strictly higher demand, or 0 if none follows.

Constraints: Strictly higher means greater than, not equal. A day with no higher day after it reports 0. Values may repeat.

Formula Guidance

```
while the stack is non-empty and  $a[i] > a[\text{stack.top}]$ :  
     $\text{answer}[\text{top}] = i - \text{top}$  ; pop  
push  $i$ 
```

Sample Input

```
8  
73 74 75 71 69 72 76 73
```

Sample Output

```
Days To Higher: 1 1 4 2 1 1 0 0
```

⚠ **Common Pitfall:** Use a strict greater-than test so equal demands don't resolve a waiting day early, and leave unresolved days at 0.

💡 **Thinking Skill:** A monotonic stack resolves each day exactly when its first higher day arrives, so every element is pushed and popped once — the forecast is linear, not quadratic.

Level 5 — Circular Event Buffer

PrimeFlow keeps only the most recent C events in a fixed ring buffer. When a new event arrives and the buffer is full, it overwrites the oldest one and advances the head — so the log always shows the latest C events in arrival order.

Concepts Covered

- A fixed-size ring buffer
- Overwriting the oldest entry when full
- A head index plus a size with modular arithmetic
- Reading out oldest-to-newest

Formula Guidance

```
write slot = (head + size) mod C
if full: head = (head + 1) mod C ; overwrites++
else:   size++
```

Sample Input

```
3
5
LOG 1
LOG 2
LOG 3
LOG 4
LOG 5
```

Problem Statement

Given: A capacity C , a count n , then n LOG operations, each with an event id.

Required output: The buffer contents from oldest to newest, and the number of times an old event was overwritten.

Constraints: The buffer holds at most C events. A LOG into a full buffer overwrites the oldest event and counts an overwrite. The output lists the live events oldest first.

Sample Output

```
Buffer: 3 4 5
Overwrites: 2
```

⚠ **Common Pitfall:** When full, advance the head as you overwrite so the dropped event is the true oldest and the read-out still begins there. This contrasts with a reject-when-full queue.

💡 **Thinking Skill:** A ring buffer bounds memory to C slots no matter how many events stream in; modular arithmetic makes 'oldest out, newest in' free of any shifting.

Level 6 — Robot Scheduler

PrimeFlow's fulfillment robots share processing time fairly. Each task gets a fixed time quantum; if it isn't finished, it goes to the back of the queue and waits its next turn. Report the order tasks finish and how many quantum slices it took.

Concepts Covered

- Round-robin scheduling with a queue
- A fixed time quantum
- Re-enqueuing unfinished work
- Recording the completion order

Problem Statement

Given: A task count n and a quantum q , then n service times (tasks numbered $1..n$).

Required output: The order in which tasks complete (1-based task numbers), and the total number of quantum slices executed.

Constraints: Tasks start in the queue in order $1..n$. Each turn runs $\min(\text{remaining}, q)$; if work remains the task re-enters at the back. A turn counts as one slice even if it finishes the task.

Formula Guidance

```
run = min(remaining, q)
remaining -= run ; turns++
if remaining == 0: completed ; else re-enqueue
```

Sample Input

```
3 3
5 3 8
```

Sample Output

```
Completion Order: 2 1 3
Turns: 6
```

⚠ **Common Pitfall:** Count a slice on every turn, including the one that finishes a task, and re-enqueue only when time still remains.

💡 **Thinking Skill:** A queue makes round-robin fairness automatic: finishing a slice and going to the back is just dequeue-then-enqueue, so every task keeps getting equal turns.

Level 7 — Warehouse Pathfinder

A PrimeFlow warehouse robot must reach a shelf by the shortest route across a grid of aisles and obstacles. **Breadth-first search** from the start, expanding outward level by level with a queue, finds the fewest steps to the goal.

Concepts Covered

- Breadth-first search with a queue
- Grids as implicit graphs
- Level-by-level expansion for shortest paths
- Tracking visited cells

Problem Statement

Given: Grid dimensions R and C, then R rows. 'S' is the start, 'E' the goal, '#' an obstacle and '.' free floor.

Required output: The fewest steps from S to E, or -1 if E is unreachable. Whether the goal is reachable (Yes/No).

Constraints: Moves are 4-directional (up, down, left, right). Obstacles ('#') cannot be entered, and the grid edges act as walls. Each free cell is visited at most once.

Formula Guidance

BFS distance: $\text{dist}[\text{neighbour}] = \text{dist}[\text{cell}] + 1$
the queue expands cells in non-decreasing distance order

Sample Input

```
4 5
S....
.##..
..#..
....E
```

Sample Output

```
Steps: 7
Reachable: Yes
```

⚠ **Common Pitfall:** Mark a cell visited when you enqueue it, not when you dequeue it, or the same cell can be queued many times and the search slows badly.

💡 **Thinking Skill:** BFS explores in rings of equal distance, so the moment it touches the goal it has found the shortest path — nothing can be shorter than the ring that reached it first.

Level 8 — Delivery Reach

PrimeFlow models its delivery network as a graph of hubs and routes. From the central hub, **breadth-first search** finds how many hops it takes to reach every node; a **stack** then rebuilds the route to the farthest reachable hub by walking the BFS parents backward.

Concepts Covered

- BFS over an explicit graph
- Hop distances from a source
- Parent pointers for path reconstruction
- Reversing a parent chain with a stack

Problem Statement

Given: A node count N and edge count M, then M undirected edges u v. The hub is node 0.

Required output: How many nodes are reachable from the hub (including itself), the maximum hop distance, the farthest node (smallest id on a tie), and the hop-by-hop path from the hub to that farthest node.

Constraints: Edges are undirected; neighbours are explored in ascending id order for a deterministic path. Hop distance counts edges from the hub. On a tie for the maximum distance, the smallest node id wins.

Formula Guidance

BFS: $\text{dist}[w] = \text{dist}[u] + 1$ and $\text{parent}[w] = u$
path: follow parent from the farthest node back to the hub, then reverse

Sample Input

```
6 6
0 1
0 2
1 3
2 3
3 4
4 5
```

Sample Output

```
Reachable: 6
Max Hops: 4
Farthest Node: 5
Path: 0 -> 1 -> 3 -> 4 -> 5
```

 **Common Pitfall:** Rebuild the path by reversing the parent chain with a stack (farthest → hub becomes hub → farthest); printing the parents in visit order would give the path backwards.

 **Thinking Skill:** This capstone pairs the two structures directly: a queue drives the level-order search that measures distance, and a stack reverses the parent chain into a forward path.